

Investigación02

APLICACIONES DE ESCRITORIO
EN

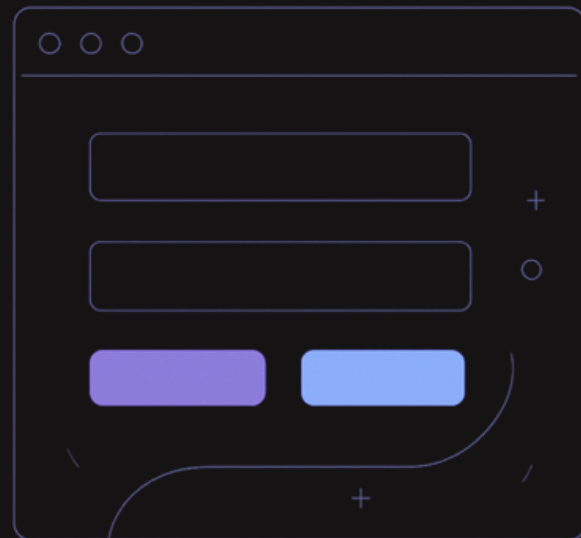
.NET

Arquitectura, integración de APIs y patrones de sincronización de datos en
ecosistema Windows



Fundamentos

WINDOWS FORMS: DEFINICIÓN Y POSICIONAMIENTO



WinForms es el marco clásico de .NET para interfaces gráficas en escritorio, operando como wrapper sobre la API nativa de Windows (User32 y GDI+).

- **Frente a WPF:** Usa renderizado vectorial con DirectX (GPU) y separa diseño (XAML) de lógica (MVVM); WinForms depende de CPU y mezcla diseño con código imperativo.
- **Frente a .NET MAUI:** Es multiplataforma (Windows, macOS, Android, iOS); WinForms está restringido exclusivamente a Windows.

Fundamentos

ARQUITECTURA DE EVENTOS Y CONTROLES

MESSAGE LOOP

Hilo UI escucha acciones del SO

CICLO DE EVENTOS WINFORMS

EVENTOS Y DELEGADOS

Se traducen a EventHandler en code-behind

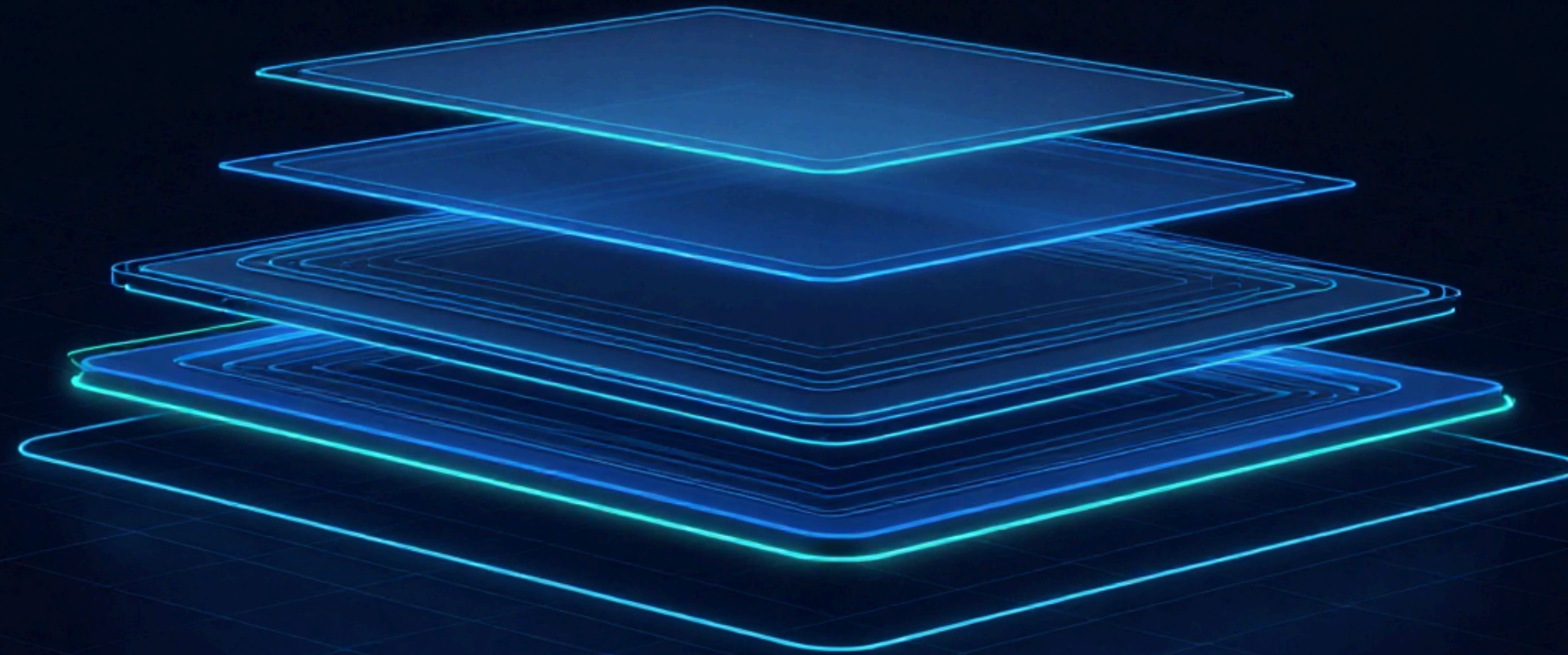
CONTROLES HWND

Cada control encapsula un identificador nativo



Fundamentos

WINFORMS VS. DESARROLLO WEB (ASP.NET)



VENTAJAS DE WINFORMS

- **Acceso nativo:** Comunicación directa con hardware (puertos serie, escáneres) y sistema de archivos local.
- **Stateful:** Reside en RAM, sin pérdida de estado propia de HTTP.
- **Offline:** Opera sin conexión a internet ni servidores web.

LIMITACIONES DE WINFORMS

- **Distribución:** Requiere instalación o actualización del ejecutable en cada máquina.
- **Portabilidad:** Limitado exclusivamente a escritorio Windows.
- **Seguridad:** La conexión directa a BD (ADO.NET) puede exponer credenciales en el cliente, a diferencia del modelo servidor-cliente de ASP.NET Core.

Integración

CONSUMO DE SERVICIOS WEB REST DESDE C#



¿QUÉ ES UNA API REST?

Interfaz que conecta sistemas mediante verbos HTTP (GET, POST, PUT, DELETE) sobre recursos URL, devolviendo datos en JSON.

HTTPCLIENT — BUENAS PRÁCTICAS

Reutilizar instancia única o **HttpClientFactory**; configurar **Timeout**; verificar **EnsureSuccessStatusCode**.

SYSTEM.TEXT.JSON

Librería nativa de .NET para serializar/deserializar JSON ↔ objetos C# fuertemente tipados, sin dependencias externas.

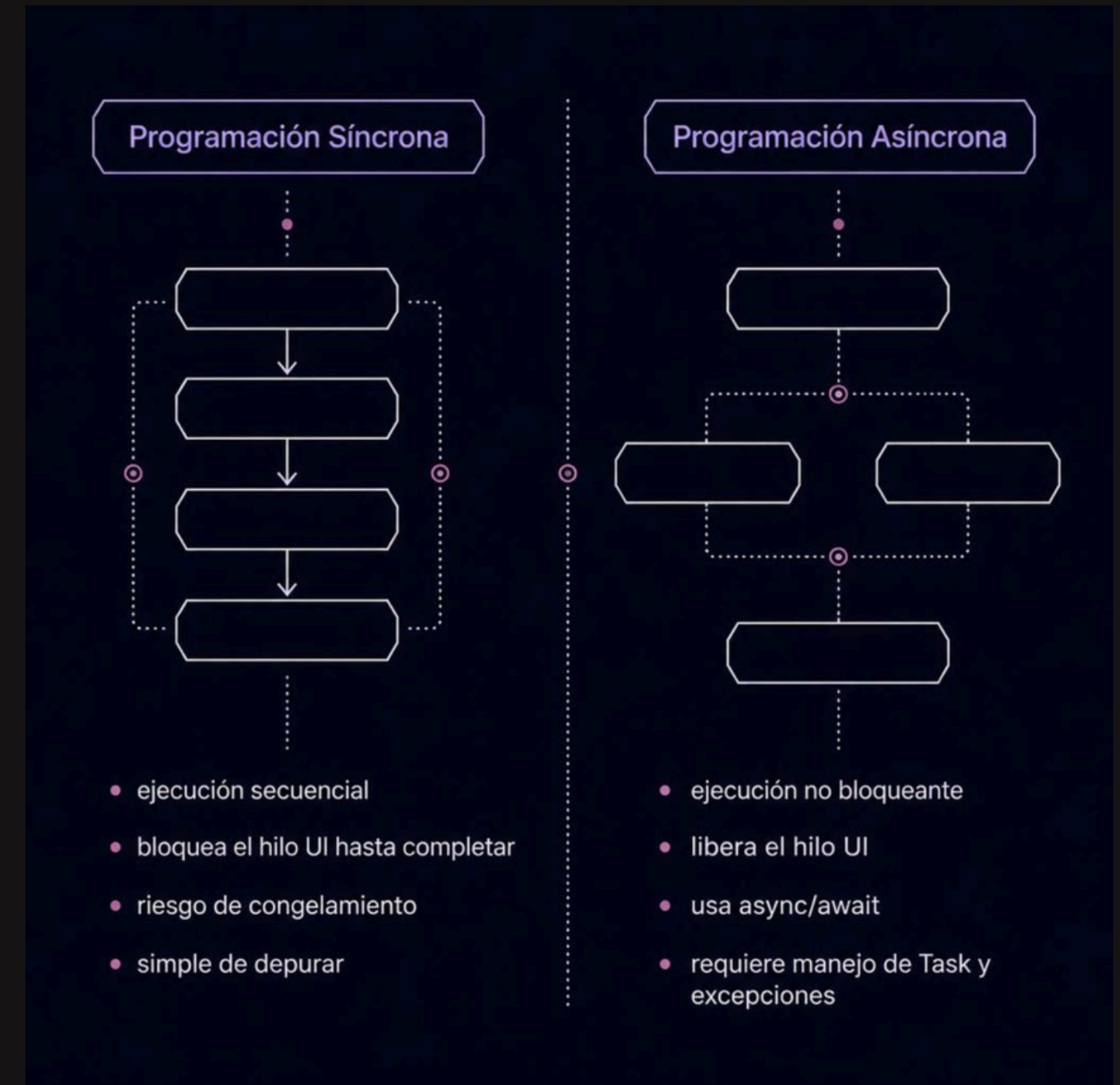
Integración

PROGRAMACIÓN ASÍNCRONA EN C#

ASYNC / AWAIT EN C#

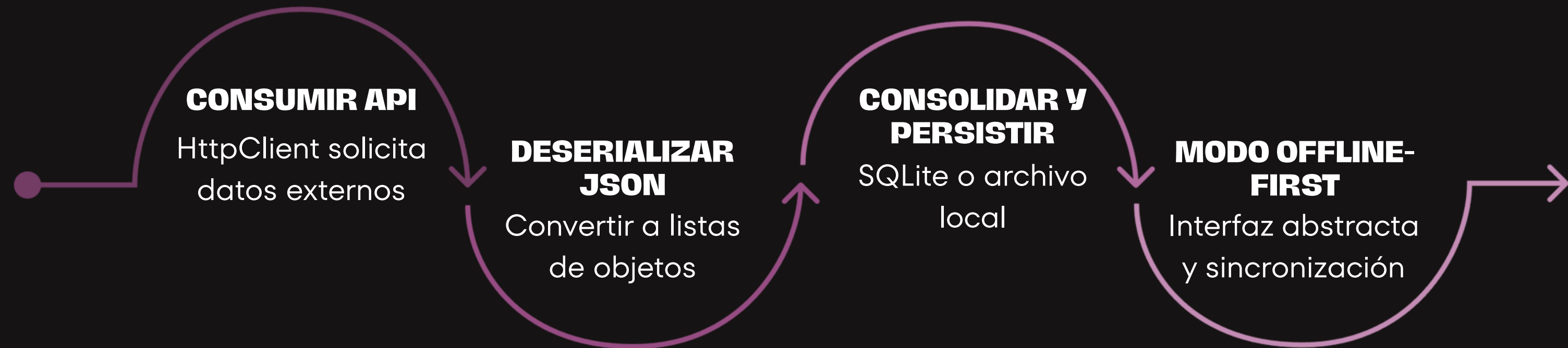
Las palabras clave `async` y `await` permiten escribir código no bloqueante con apariencia secuencial. El método marcado con `async` retorna un `Task`, y `await` suspende la ejecución hasta completar la operación sin congelar la interfaz.

En WinForms, usar **await** en eventos de UI evita el congelamiento durante peticiones HTTP o acceso a disco.



Integración

INTEGRACIÓN DE DATOS: API + ALMACENAMIENTO LOCAL



La arquitectura de sincronización combina consumo de API externa con persistencia local mediante un patrón de abstracción que desacopla la fuente remota de la capa local, permitiendo operación continua incluso ante fallos de red.



Casos de uso

CASOS DE USO EN LA INDUSTRIA

SISTEMAS POS E INVENTARIOS

Registran ventas localmente en BD liviana. Al detectar conexión, sincronizan stock con ERP/CRM en la nube vía REST.

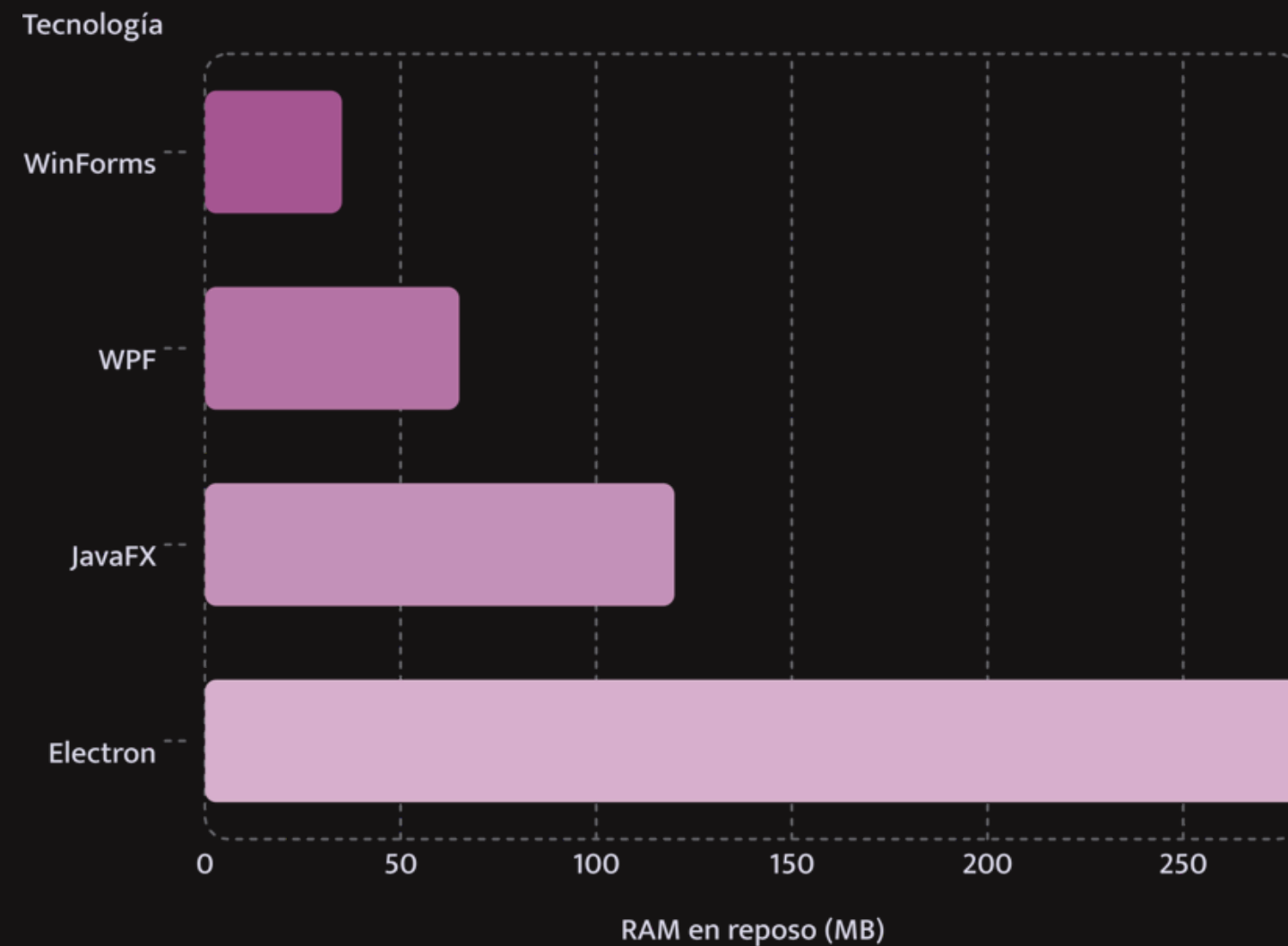
GESTIÓN DE ACTIVOS (IBM MAXIMO)

Exportan e importan grandes volúmenes mediante CSV/JSON en procesos asíncronos, consolidando datos en AWS S3.

HERRAMIENTAS DE CONSULTA

Apps WinForms que guardan favoritos/historiales en favoritos.json, permitiendo revisión persistente aunque la red falle.

Comparativa **CASOS DE USO EN LA INDUSTRIA**



CRITERIOS DE SELECCIÓN

- WinForms / WPF: Alta integración con Windows, excelente rendimiento en RAM (35–65 MB).
- .NET MAUI: Multiplataforma oficial de Microsoft (Windows, macOS, Android, iOS).
- Electron: Usa tecnologías web (Chromium + Node.js); facilita el desarrollo pero consume 280 MB en reposo.
- JavaFX: Alternativa JVM con consumo intermedio (120 MB), sin integración nativa .NET.